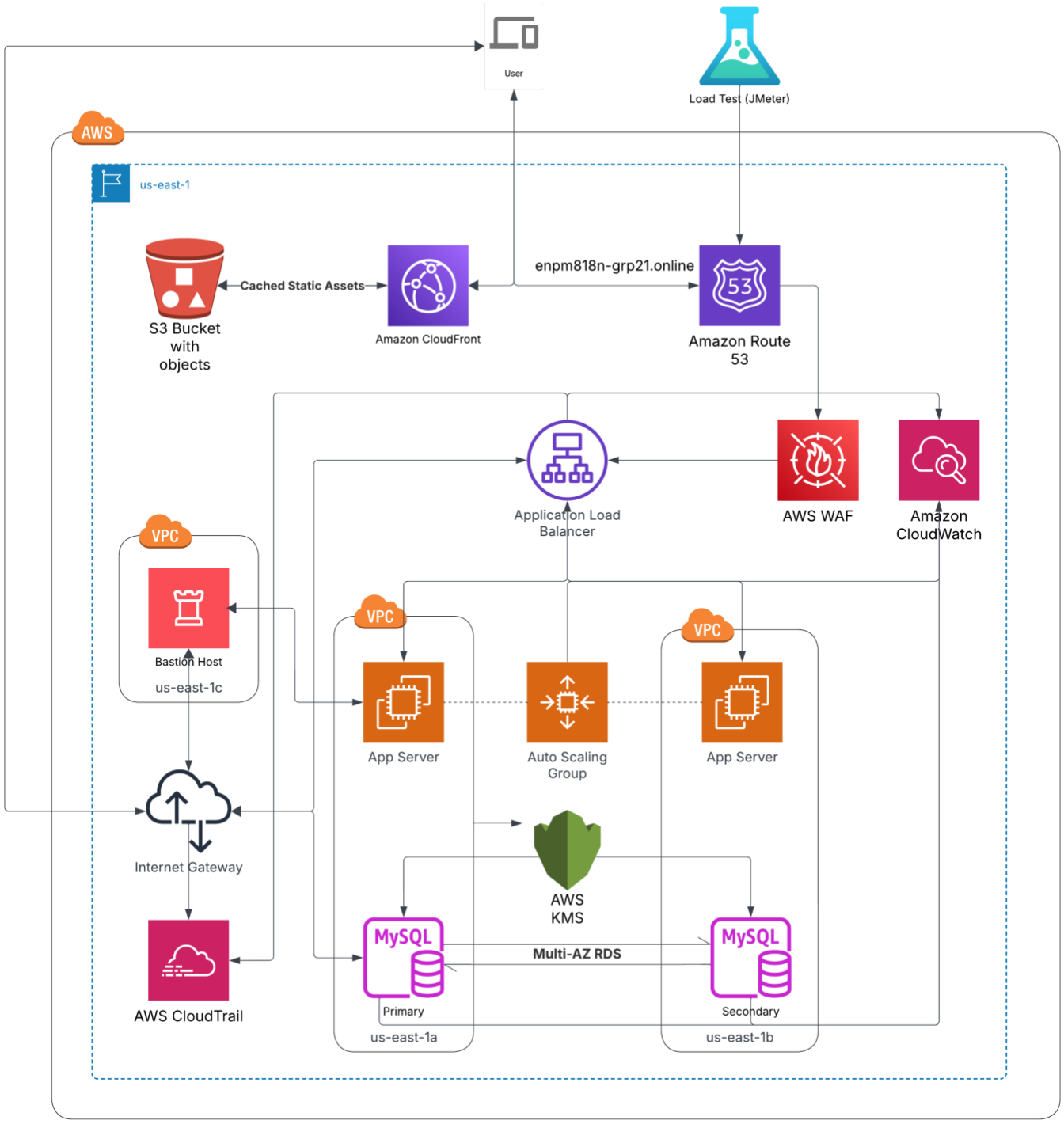
# Midterm Project

## ENPM818N – Group 21


Mahima Rameshkumar Shenoy 121413888
Nimal Kurien Thomas 121317681
Shubham Sinha 120319006
Sri Harsha Chayanulu Varahabhatla 120478345


# Table of Contents

# 1. Architecture Diagram



# 2. Phase-wise Deliverables

## a. Phase-1: EC2 Instances, Load Balancer, Auto Scaling, and RDS Setup

The successful implementation of this phase will achieve the following:

- Fully functional EC2 instances running a web server with an e-commerce application.
- Application Load Balancer (ALB) to distribute incoming traffic among EC2 instances.
- Auto Scaling Group to automatically adjust the number of instances based on CPU utilization or traffic patterns.
- RDS MySQL with Multi-AZ deployment for high availability, data encryption using AWS KMS, and database tables for products, users, and orders.
- CloudWatch Alarms configured to trigger Auto Scaling based on CPU usage.

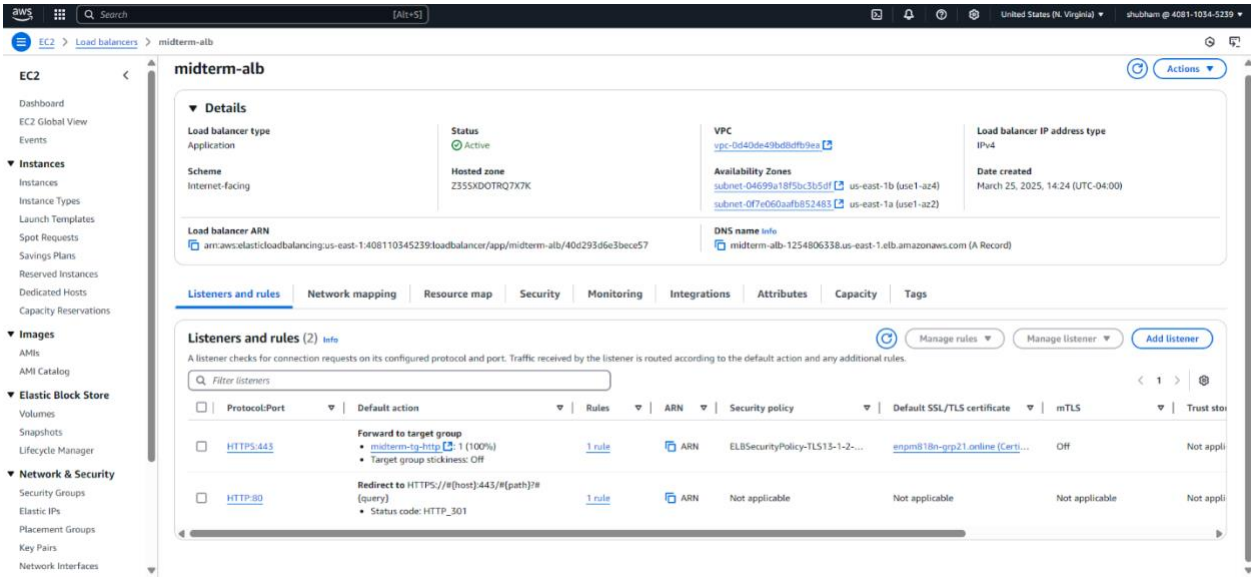
### Implementation Details

1. AMI Creation. The web server AMI is created with the following configuration:
    - Base Image: Ubuntu (Latest LTS version)
    - AMI Type: Pre-configured custom AMI
    - Installed Software:
        - Web Server: Apache (or NGINX)
        - Programming Environment: PHP, Node.js, MySQL Client
        - Security Updates: Latest OS patches and security configurations
    - Application Setup:
        - The e-commerce platform files are stored in /var/www/html/
        - Database connection settings are pre-configured to connect to Amazon RDS MySQL
2. Auto Scaling Configuration.
    - Launch Template: Defines the configuration for launching EC2 instances, including AMI, instance type, and security settings.
    - Auto Scaling Group (ASG):
        - Automatically increases or decreases the number of EC2 instances based on predefined conditions.
        - Ensures at least one instance is always running (minimum size).
        - Restricts the maximum number of instances to three (scalable if required).
    - Scaling Policies:
        - Scaling is triggered based on CPU utilization exceeding a 50% threshold.
        - Additional instances are launched when traffic spikes.
        - When demand decreases, extra instances are terminated to optimize costs.
3. RDS MySQL Setup.
    - Database Engine: MySQL is used for managing the backend data.
    - Multi-AZ Deployment: The database is deployed in multiple availability zones to ensure high availability and automatic failover in case of an outage.
    - Security Features:

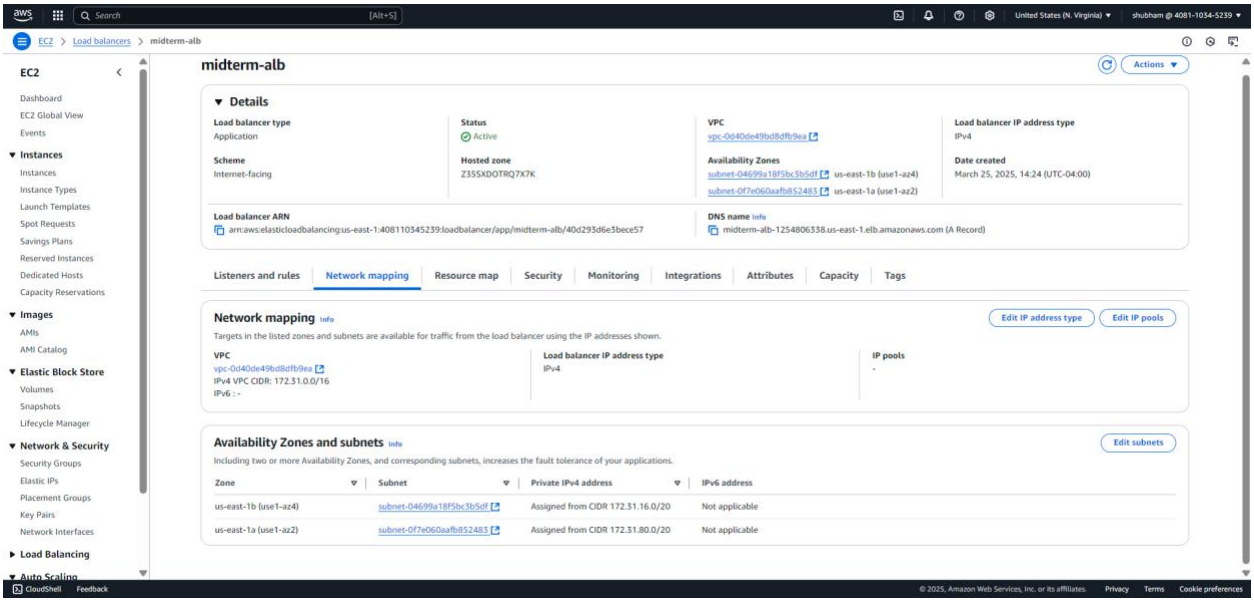
        - Amazon KMS encryption ensures that stored data is encrypted at rest.
        - A dedicated security group is configured to allow access only from EC2 instances.
    - Database Schema:
        - The database includes tables for managing users, products, and orders.
        - Supports basic CRUD (Create, Read, Update, Delete) operations for handling transactions and user interactions.
4. ALB Setup.
    - Application Load Balancer (ALB) deployed across multiple Availability Zones for high availability.
    - Security Group: Allows inbound traffic on port 443 (HTTPS) only.
    - VPC & Subnets: Configured within a designated VPC, spanning multiple subnets for redundancy.
    - Target Group Type: Instance-based, forwarding traffic to EC2 instances.
    - Listener: HTTPS (Port 443). Uses AWS Certificate Manager (ACM) for SSL/TLS encryption.
    - Routing: Default rule routes traffic towards the target group.
5. CloudWatch Monitoring and Alerts.
    - Database Engine: MySQL is used for managing the backend data.
    - Multi-AZ Deployment: The database is deployed in multiple availability zones to ensure high availability and automatic failover in case of an outage.
    - Security Features:
        - Amazon KMS encryption ensures that stored data is encrypted at rest.
        - A dedicated security group is configured to allow access only from EC2 instances.
    - Database Schema:
        - The database includes tables for managing users, products, and orders.
        - Supports basic CRUD (Create, Read, Update, Delete) operations for handling transactions and user interactions.

## b. Phase-2: Security Configurations

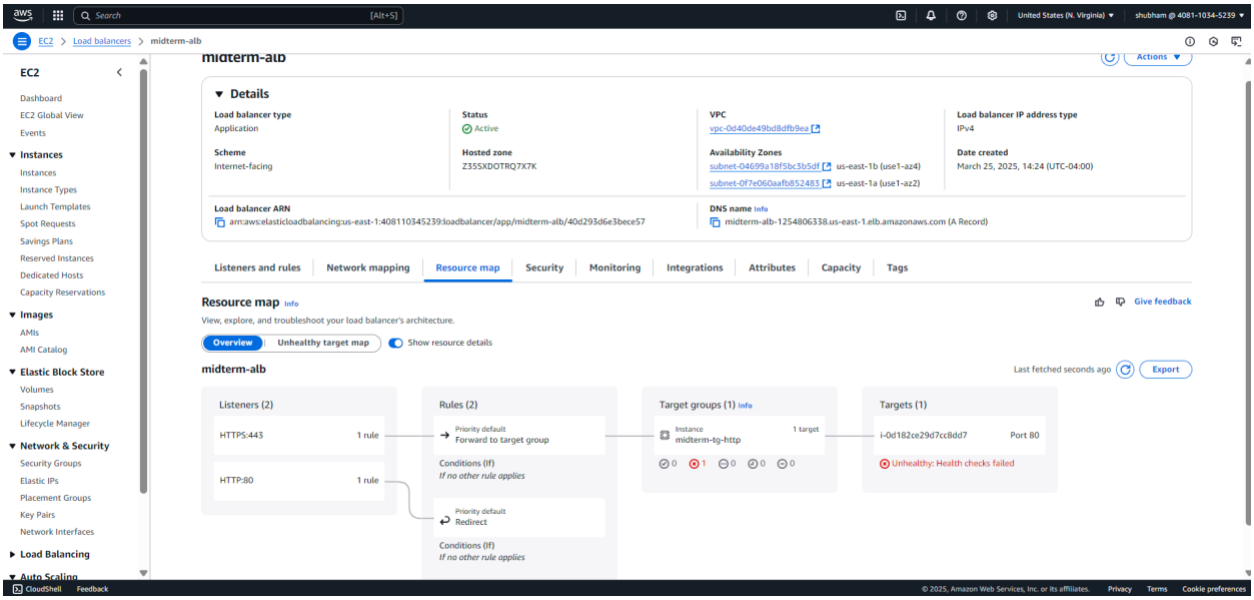
1. The image below depicts the midterm-alb, which is aimed to improve the availability and performance of our e-commerce platform by equally distributing incoming traffic. It runs in two Availability Zones (us-east-1a and us-east-1b), which improves availability and resilience by routing traffic between them.



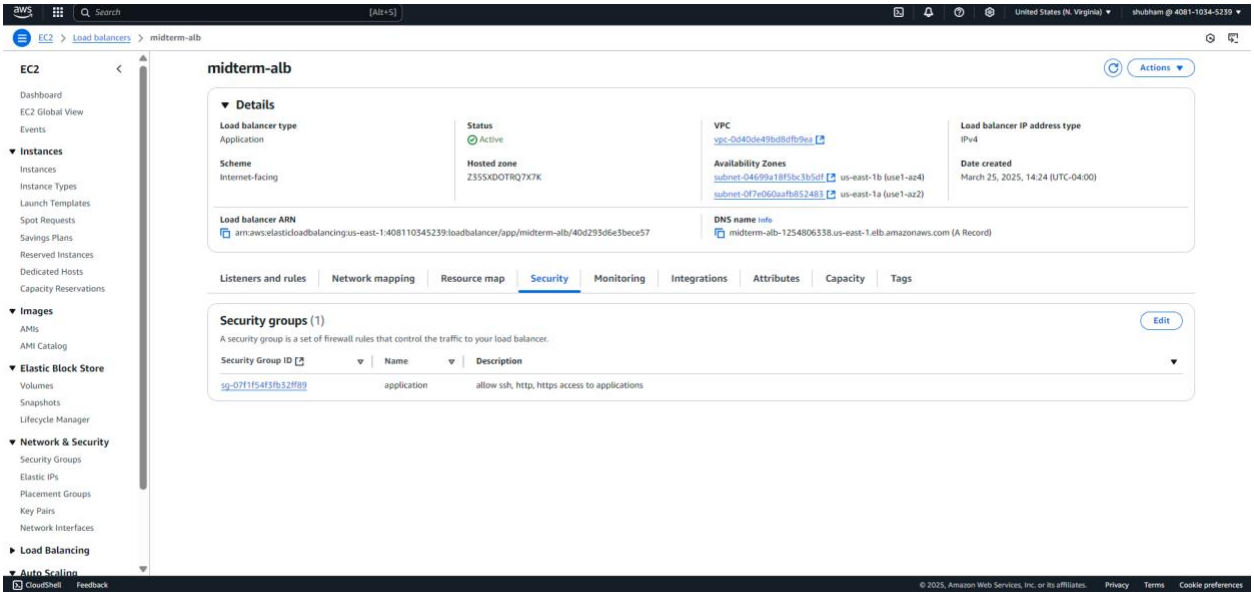

2. The load balancer has two listeners: HTTP on port 80 and HTTPS on port 443. The HTTP listener capture and redirect unencrypted requests to HTTPS with a 301 status code, ensuring all communication is safe. ▪ The HTTPS listener routes encrypted requests to the midterm-tg-http, which distributes them among EC2 instances in the Auto Scaling Group. The HTTPS listener is configured with SSL/TLS certificates for the domain enpm818n-grp21.online, ensuring data privacy and integrity. Furthermore, a security policy (ELBSecurityPolicy-TLS13-1-2-2021-06) enforces secure protocols to provide a secure connection for all user traffic. The load balancer, with its dual listener setup and security restrictions, provides a reliable way to handle user traffic.




3. The load balancer works in an internet-facing approach within the vpc-0d40de49bd8dfb9ea VPC, which has an IPv4 CIDR block of 172.31.0.0/16. This configuration employs two Availability Zones—us-east-1a and us-east-1b—each with a specific public subnet (172.31.80.0/16 for us-east-1a and 172.31.16.0/16 for us-east-1b).

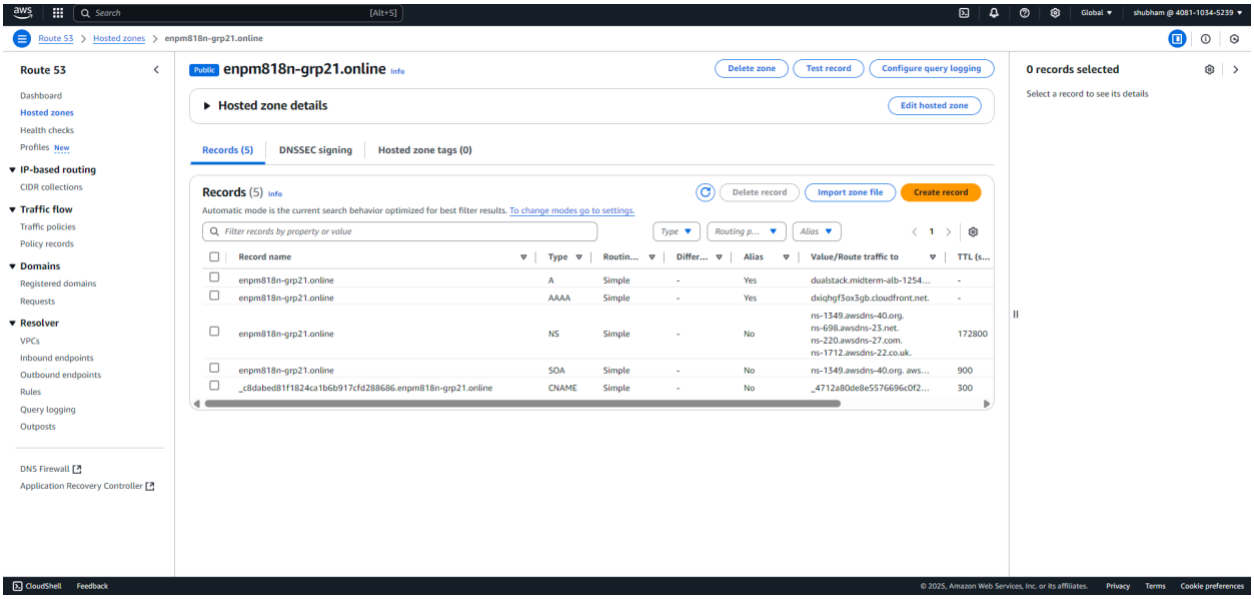



4. The HTTPS listener routes encrypted traffic to a specific target group, mideterm-tg-http, which includes one registered EC2 instance on port 80.



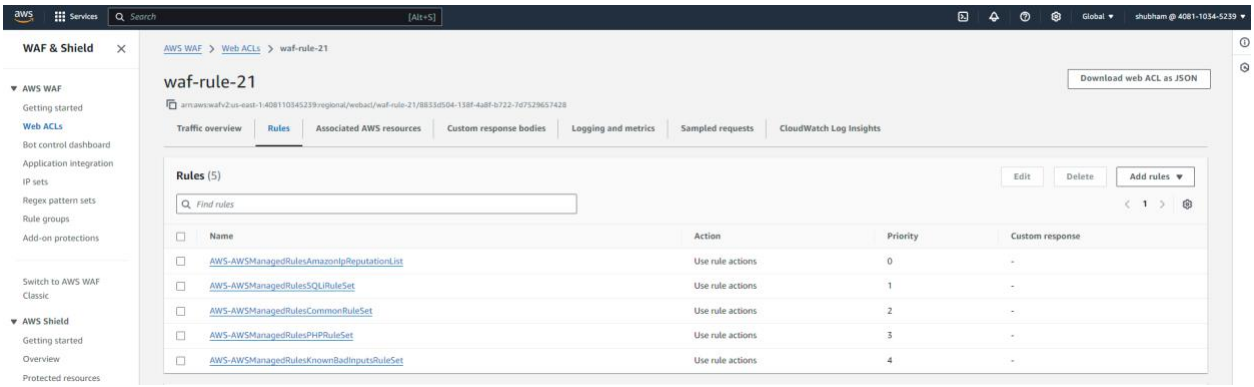

5. The midterm-alb is protected by a specific security group (sg-07f1f54f3fb32ff89), which is set to accept SSH, HTTP, and HTTPS connections. This security group is responsible for controlling and filtering inbound traffic, ensuring that only web traffic on ports 80 (HTTP) and 443 (HTTPS) can reach the load balancer and SSH on port 22.

6. Route 53
    - AWS provides Route53, a service that allows us to handle DNS for our domain. This is accomplished by simply referring the domain to the AWS nameservers specified in the hosted zone.

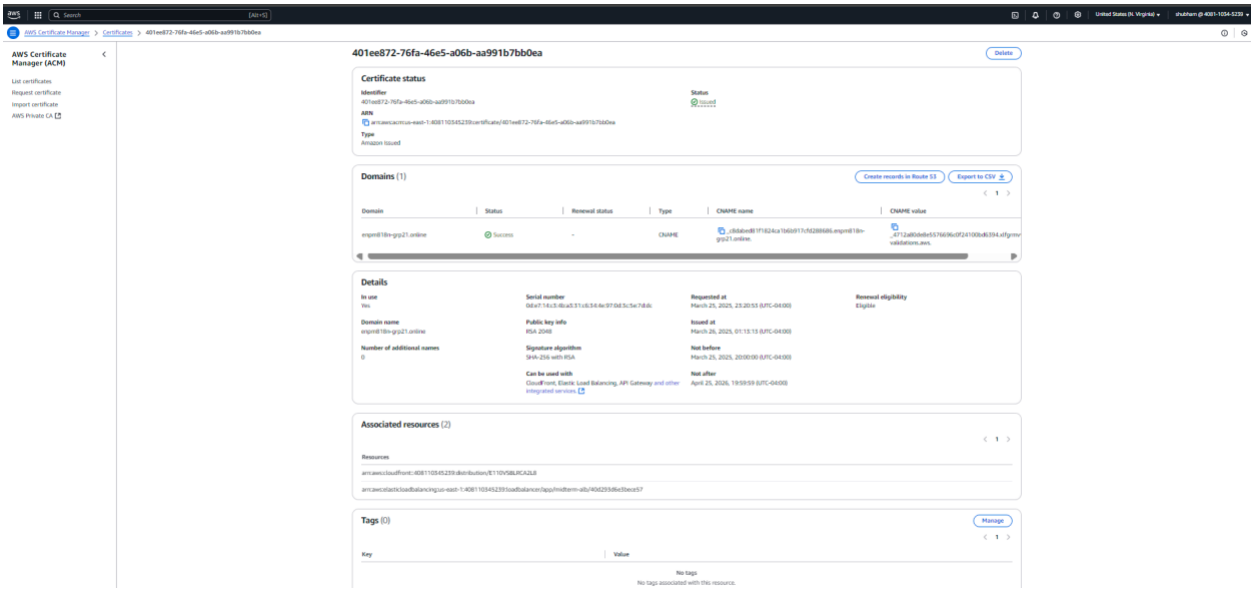



7. Additionally an A record pointing to the Load Balancer and a CNAME record needed to issue the SSL certificate for the domain.

8. Waf:
    - The WAF configuration for "waf-rule-21" contains a large number of managed rule groups, including the AWSManagedRulesAmazonIpReputationList, SQLiRuleSet, CommonRuleSet, PHPRuleSet, and KnownBadInputsRuleSet. With a Web ACL capacity consumption of 1225 out of 5000 WCUs, the configuration is well within restrictions, optimizing security without sacrificing performance. The default action is set to "Allow" for requests that do not fit any specified criteria, ensuring a balanced security approach while allowing required traffic.




9. Security Measures:
    - SSL for domain: According to our architecture's process, when our domain enpm818n-grp21.online is accessed, Route53 routes the request through Cloudfront and WAF before sending it to the Load Balancer. Because our website is an e-commerce site, sensitive data, such as login information, order details, payment information, and so on, must be encrypted while in transit. To do this, we enabled an SSL certificate from AWS Certificate Manager (ACM) on our domain, which is also related with the Load balancer and CloudFront services we utilized.

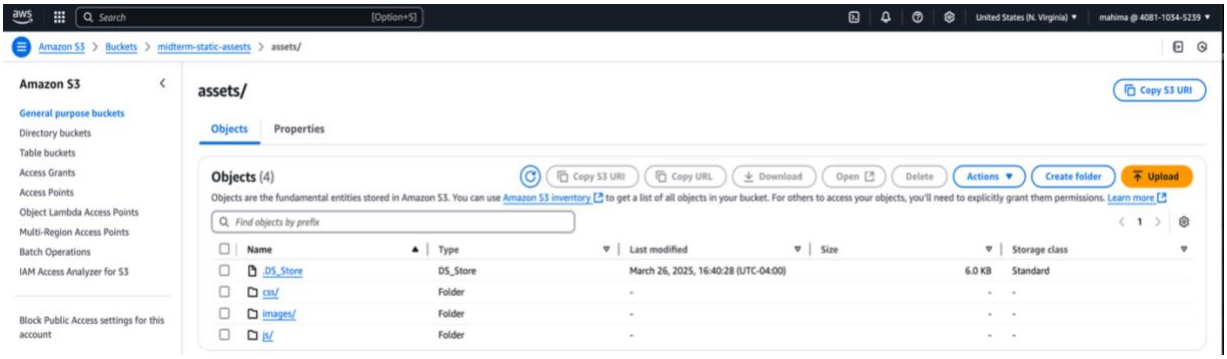


## c. Phase-3: CloudFront Setup and Performance Tuning

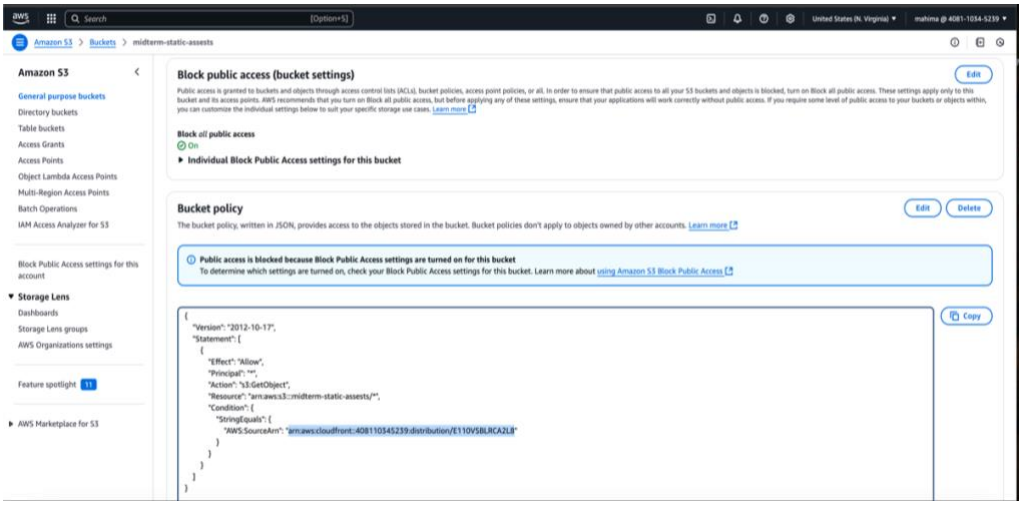
AWS CloudFront was used to cache and deliver static assets such as CSS, JavaScript, and images, improving website performance. By serving content from the nearest edge location, it reduces load times and decreases latency. This setup optimizes content delivery, minimizes the load on the origin server, and enhances the user experience. With a globally distributed network, CloudFront ensures fast and reliable access to static resources.

Setup:

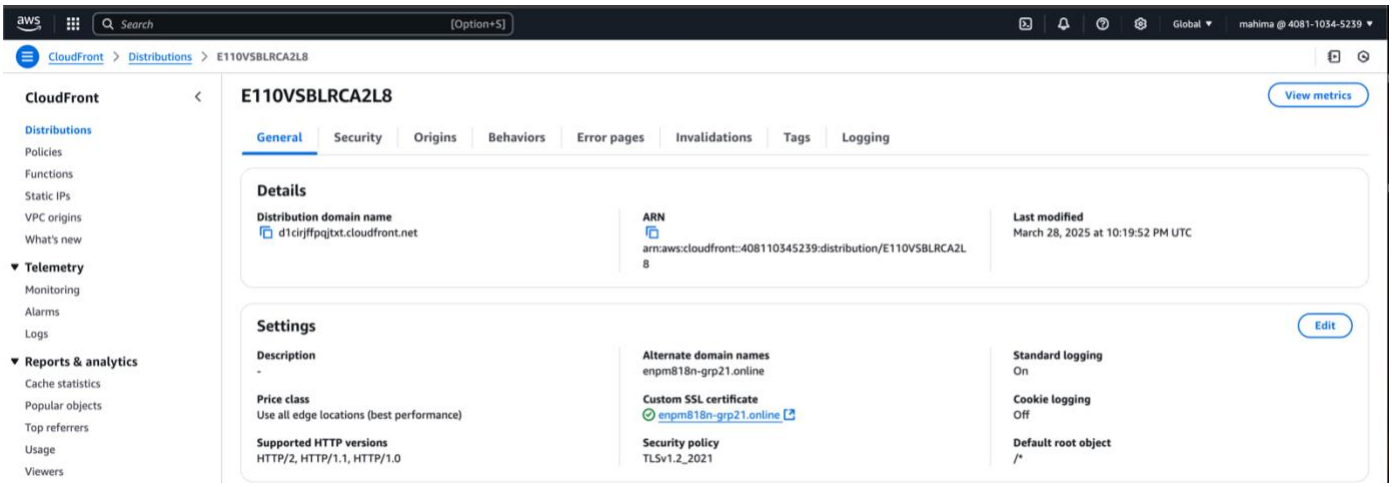
1. Creation of S3 Bucket for storing Static Assests
   - The midterm-static-assests S3 bucket was put up to act as CloudFront's origin, offering centralised storage for items such as pictures, CSS, and JavaScript. As seen by the arrangement under the "assets" directory in the S3 configuration, this structure guarantees simpler file administration and retrieval by grouping assets under specific folders—css, js, and images.

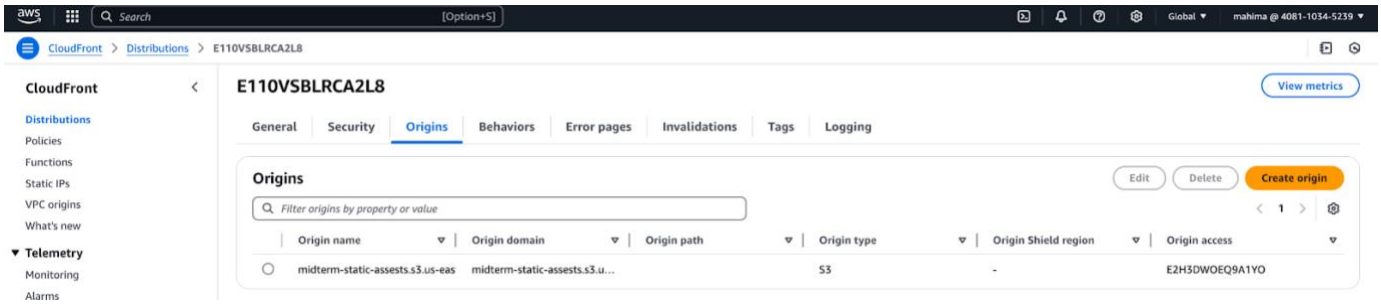



2. Enhancing Security in S3 and CloudFront Integration
   - Blocking Public Access – The S3 bucket is fully restricted from public access to prevent unauthorized usage.
   - Origin Access Control (OAC) Implementation – OAC ensures that the S3 bucket is accessible only through CloudFront, enhancing security.
   - Controlled Access via CloudFront – A bucket policy allows only the designated CloudFront distribution to retrieve objects by granting permission for the s3:GetObject action, while restricting direct access.

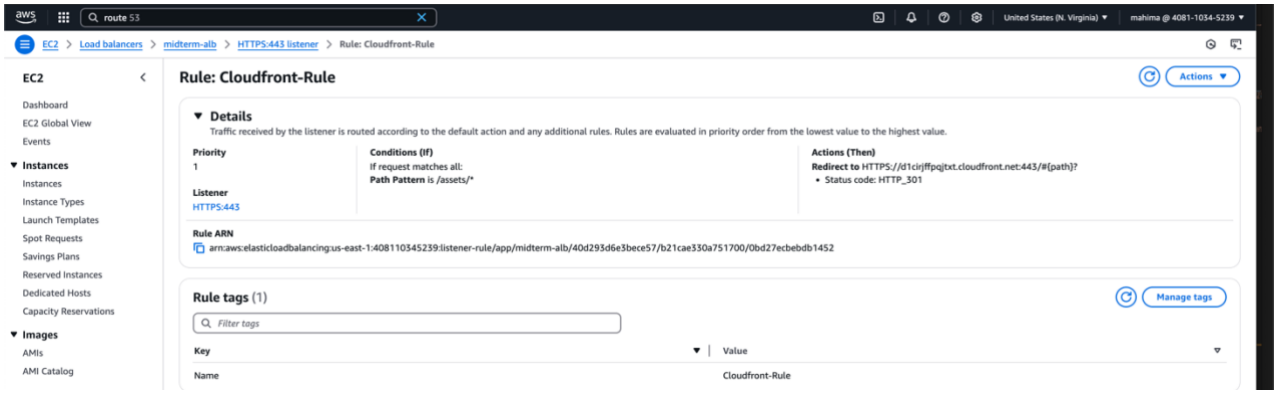


3. CloudFront Distribution
    - To improve load speeds by retrieving assets from edge locations closest to users, a CloudFront distribution E110VSBLRCA2L8 was created to cache and distribute content globally.

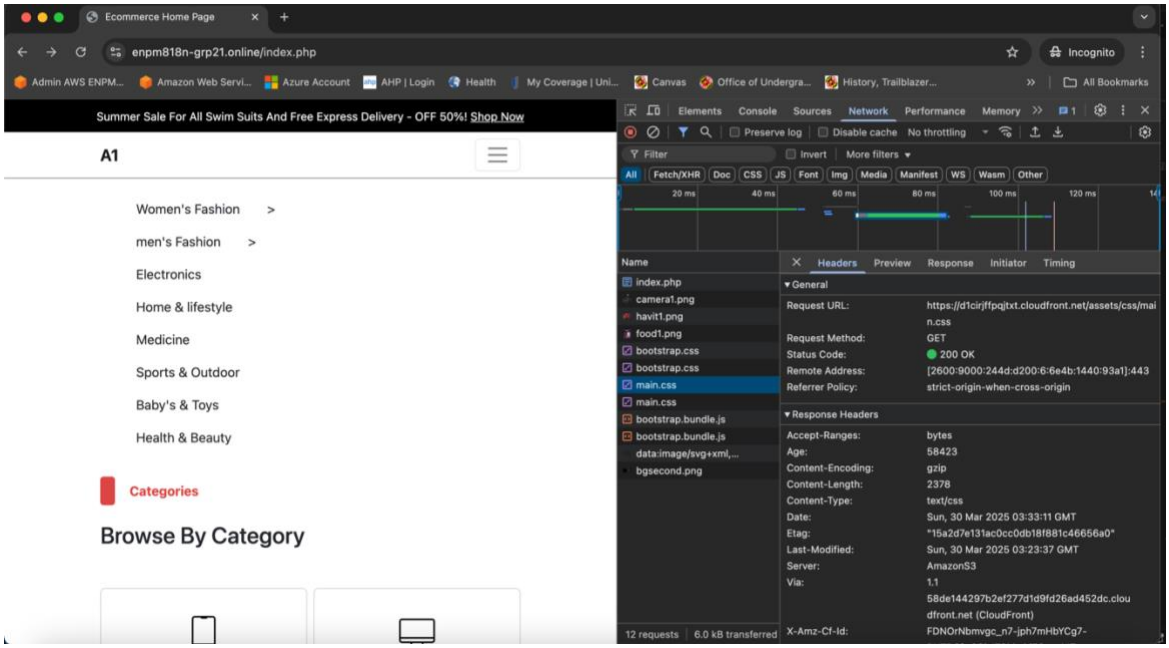


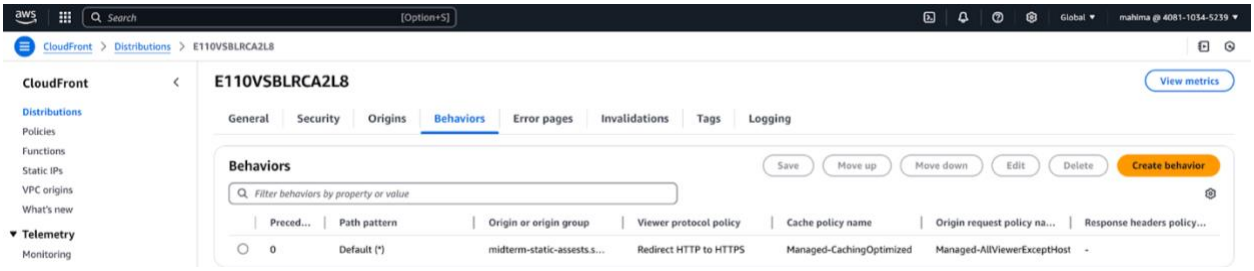

4. Redirecting to CloudFront URL.
    - To leverage the benefits of the CDN, the Application Load Balancer (ALB) is configured with a listener rule that matches requests to the /assets/* path and redirects them to the CloudFront distribution's URL (https://d1cirjffpqjtxt.cloudfront.net).
    - **How It Works:**
      **--> Client Request:** A user requests a resource, such as https://enpm818n-grp21.online/assets/images/bg.png
      **--> ALB Evaluation:** The ALB evaluates the incoming request against its listener rules. Upon matching the path condition /assets/*, it triggers the redirect action.
      **--> Redirect Response:** The ALB responds with an HTTP 301 or 302 redirect status, instructing the client's browser to fetch the requested resource from the specified CloudFront URL, for example, https://d1cirjffpqjtxt.cloudfront.net/assets/images/bg.png
      **--> Client Follows Redirect:** The client's browser follows the redirect to the CloudFront URL to retrieve the asset.



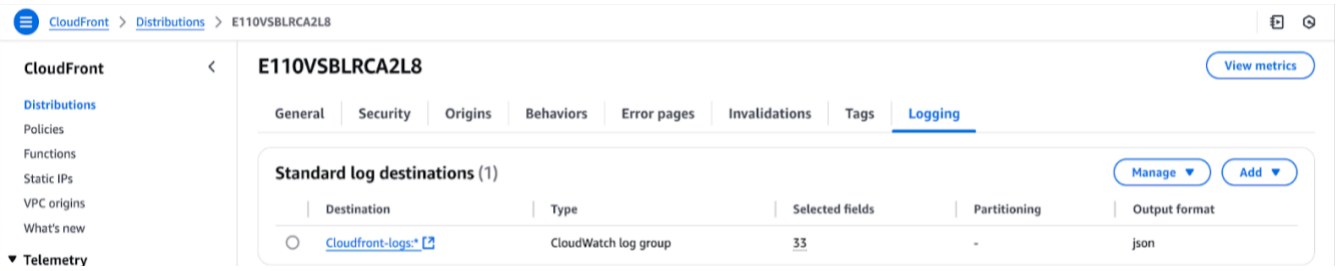

5. Performance Tuning by GZIP Compression:
   - The below images illustrates how GZIP compression is successfully applied to static assets in our web application, focusing on the main.css file that is compressed and distributed using **CloudFront**. As indicated by the **Content-Encoding: gzip** response header, CloudFront compresses the CSS file with GZIP, which is critical for optimising web speed.

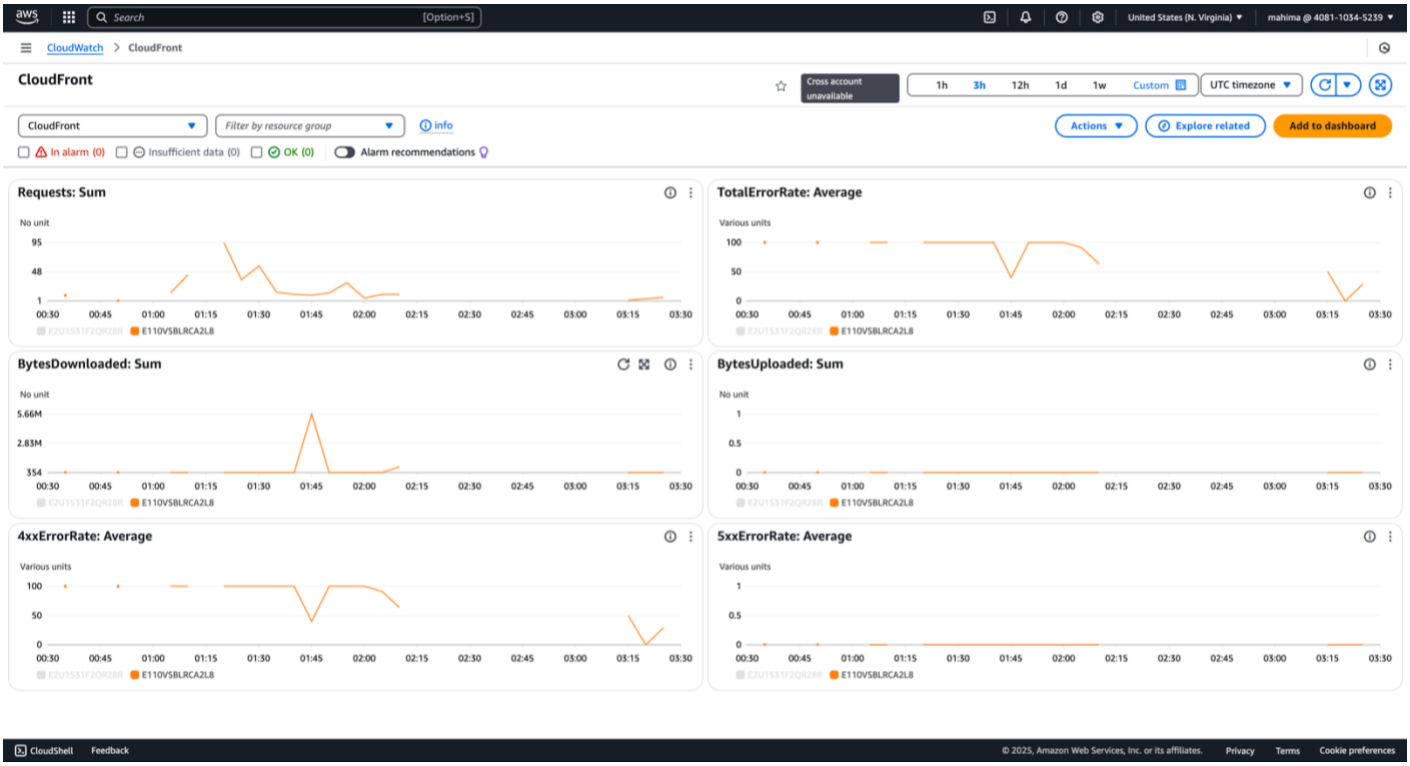



6. Content Caching:
   - Applied The "Managed-CachingOptimized" policy helps in caching objects, which reduces the load on the S3 bucket by serving frequently accessed assets directly from the cache (CloudFront edge locations) instead of making repeated requests to the S3 bucket.




7. CloudWatch Logs:
   - These are CloudWatch logs for CloudFront, providing detailed insights into the performance and usage of the distribution. They help monitor request patterns, cache hit rates, and detect any potential issues in content delivery.



### d. Phase-4: Stress Testing, CloudWatch, Cost Analysis

#### Objectives:

- Simulate very high user load to test performance limits of an instance in AWS
- Verify Auto Scaling functionality of an EC2 based on thresholds of the CPU usage
- Monitor backend services like the MySQL RDS database and its behaviour under concurrent access and heavy queries
- Validate our CloudWatch dashboard's visibility across metrics we deemed critical like CPU utilization and such
- Confirm events are captured via CloudTrail which are both infrastructure and management driven
- Analyze AWS billing trends with the help of Cost Explorer and provide cost optimizations

This involved the development and execution of a stress test that checked both application layer (web and API) and data layer (RDS), and observed infrastructure behavior for an extended time window.

#### Infrastructure:

The core architecture consists of multiple layers such as:

- Application Layer- EC2 instances in an Auto Scaling Group behind an Application Load Balancer. The instances run an e-commerce web application built on Node.js with Apache, using a custom AMI which is essentially a bootstrap.
- Data Layer: An Amazon RDS like a MySQL instance with Multi-AZ configuration for high availability can also provision data durability and encryption.
- Load Distribution: AWS load balancers can route incoming traffic to healthy targets across different zones.
- Monitoring: AWS CloudWatch and AWS CloudTrail are some great tools for system metrics, logs, and auditing.

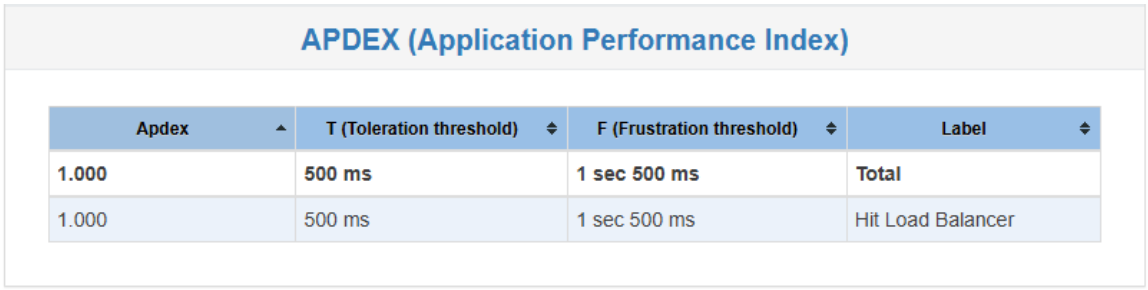
#### Stress Testing Strategy

- Tools and Configuration: Apache JMeter was chosen due to its reliability in generating HTTP traffic. A test plan was written in.jmx format, which outlined several thread groups (virtual users) with varied arrival rates and request types.
  We used ramp-up times to build the load gradually and provide Auto Scaling Group time to react. Threads were executed in loops to enable continuous activity.
- Request Types Simulated:
  - GET Requests:
    - Home page (/)
    - Product listing pages (/products, /products?category=electronics)
    - Product detail views (/products/:id)
    - Static assets (CSS, JS, images)
  - POST Requests:
    - /login: Simulating user logins

- /cart: Adding and updating cart contents
- /checkout: Finalizing purchases and creating order entries in the database

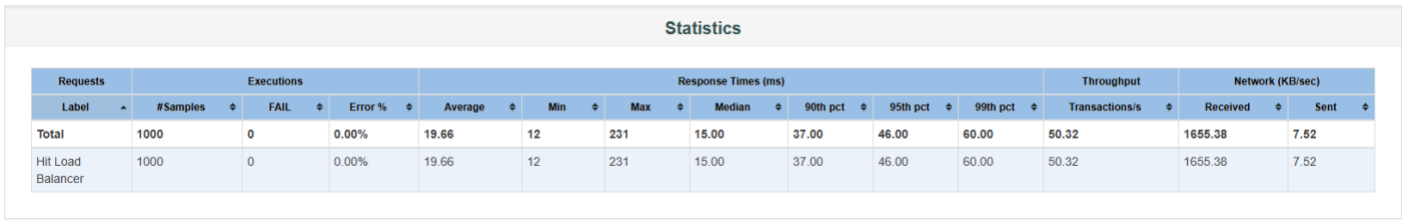
Every POST request needed backend validation and caused read/write action on the RDS database. This two-tier stress test replicated actual e-commerce behavior to fully test our infrastructure.

- Execution Scale:
  We scaled up step by step to 4,000 virtual users. Each user did 3 GETs and 2 POSTs per session cycle. That is, at full load, the system was handling up to 20,000 simultaneous HTTP requests with intensive backend database activity.
- RDS stress test:
  We created a simulated spike of activity to evaluate the performance of the RDS instance by sending a high volume of requests (`GET` and `POST`) to the endpoints that were providing the requested data from the database. We simulated concurrent database read/write behavior utilizing Apache JMeter, in order to simulate load testing under a simulated real user behavior. We were testing the RDS instance under high load and concurrent reads/writes to see how it performed. Results were extremely favorable — the APDEX score was 1.0, meaning that users are likely to be very satisfied with response times. Of the 1000 requests, the system reported zero failures, an average response time of 19.66 ms, maximum response time of 231 ms — and all 90th, 95th, and 99th percentiles latencies were under 100 ms. We also observed a steady transactional throughput lasting around 50.32 transactions per second and all transactions were returned without any errors or timeouts. The results confirmed the RDS instance and the application layer adequately supports peak load while maintaining performance reliability.



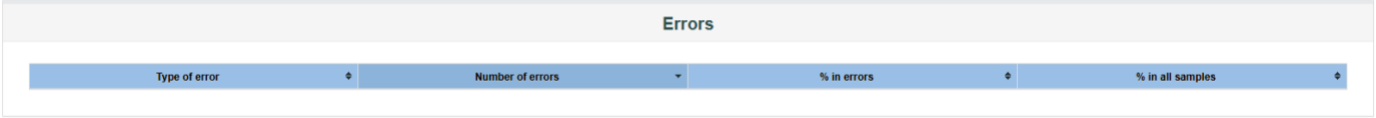
**APDEX (Application Performance Index)**

| Apdex | T (Toleration threshold) | F (Frustration threshold) | Label |
|---|---|---|---|
| **1.000** | **500 ms** | **1 sec 500 ms** | **Total** |
| 1.000 | 500 ms | 1 sec 500 ms | Hit Load Balancer |



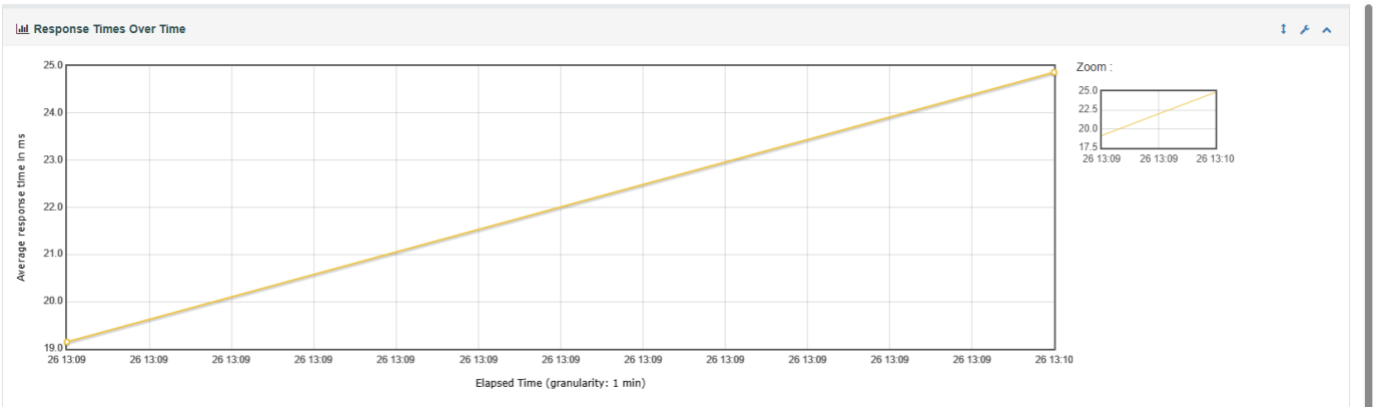
**Statistics**

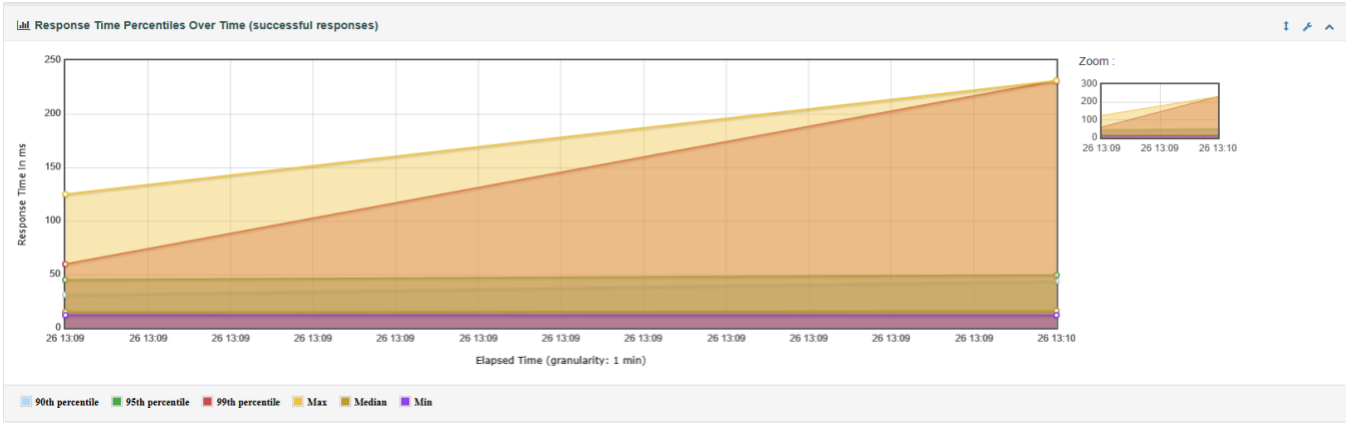
| Requests | Executions | | | Response Times (ms) | | | | | | | Throughput | Network (KB/sec) | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Label | #Samples | FAIL | Error % | Average | Min | Max | Median | 90th pct | 95th pct | 99th pct | Transactions/s | Received | Sent |
| **Total** | **1000** | **0** | **0.00%** | **19.66** | **12** | **231** | **15.00** | **37.00** | **46.00** | **60.00** | **50.32** | **1655.38** | **7.52** |
| Hit Load Balancer | 1000 | 0 | 0.00% | 19.66 | 12 | 231 | 15.00 | 37.00 | 46.00 | 60.00 | 50.32 | 1655.38 | 7.52 |

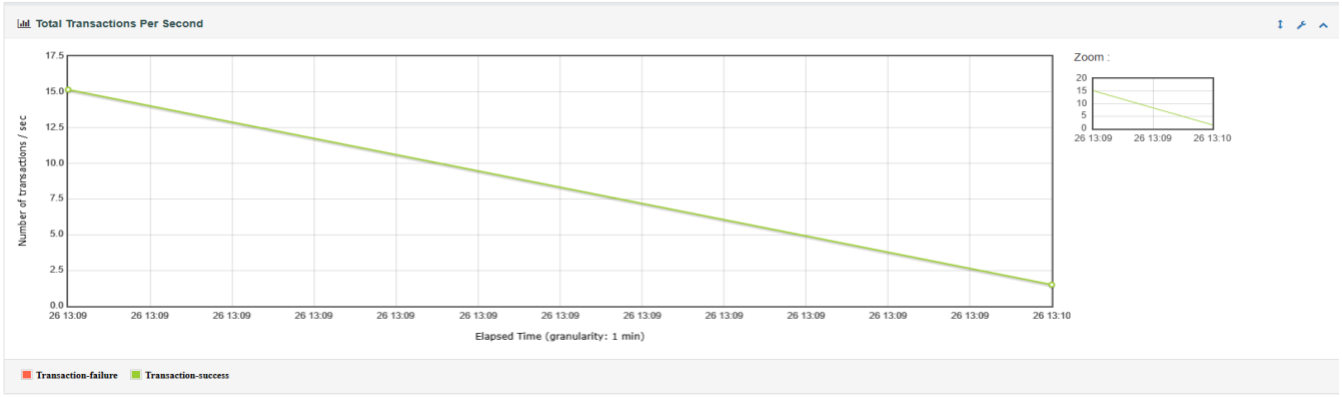


**Errors**

| Type of error | Number of errors | % in errors | % in all samples |
|---|---|---|---|





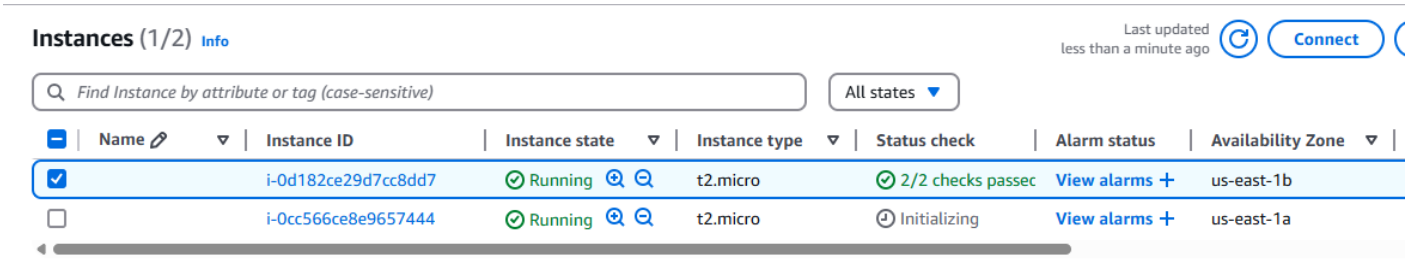
Responses occurs each time without fail

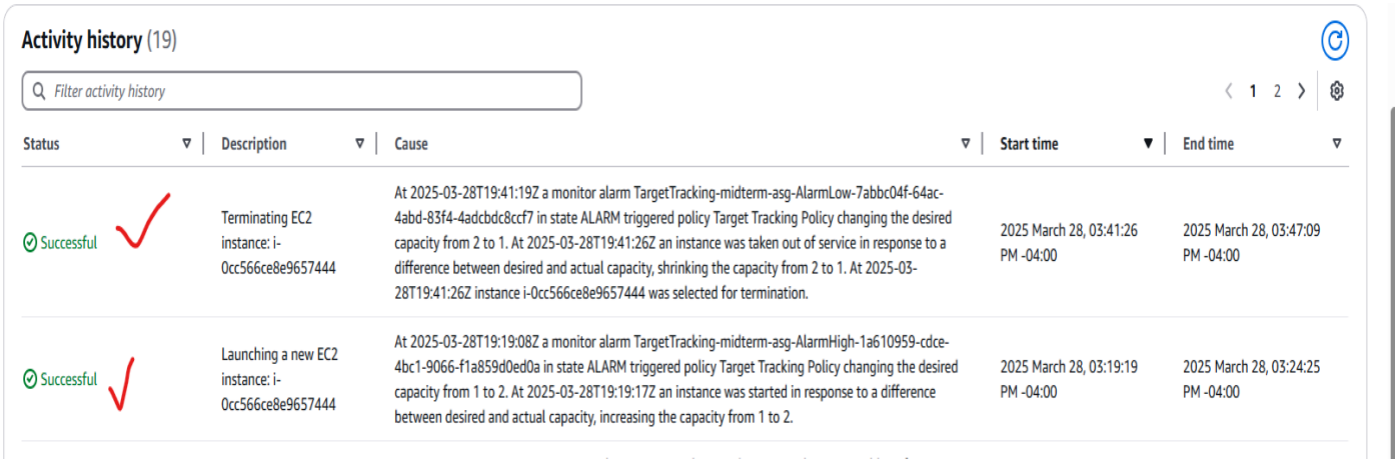
- RDS Load and Performance
  RDS performance metrics were tracked using CloudWatch and the instances were stressed and during peak POST activity, the following patterns emerged:
  - CPU Utilization increased from approximately 3-4% baseline to over 50% under heavy load
  - Database Connections spiked from 5 to 60 concurrent sessions
  - Disk Queue Depth showed sudden brief spikes during huge bursts of write queries
  - Read/Write Latency remained within thresholds that were acceptable, indicating no blocking of any kind.

  Overall, the RDS instance was responsive and stable, vindicating our instance type and provisioning configuration selection. Multi-AZ redundancy offered failover capacity even during heavy load
- Auto Scaling Configuration and Behavior:
  Target Tracking Policy
  Our ASG was configured with the following:
  - Metric: Average CPU utilization
  - Target: 50%
  - Minimum instances: 1
  - Maximum instances: 3
  - Warm-up time: 300 seconds
  - Scale-in enabled

  This setup enabled the platform to react quickly to spiked load, but avoided aggressive back-and-forth scaling by providing a grace period for instances to register into the load balancer and contribute significantly to the load.
  Scaling Events Observed:
  During testing:
  - CPU utilization crossed to about 60%, this in turn started triggering scale-out
  - Second and even a third EC2 instances launched and registered, overkill for the stress test.
  - Request traffic was distributed by ALB evenly to all instances
  - Upon completion of the test, traffic had dropped, and idle instances were scaled in

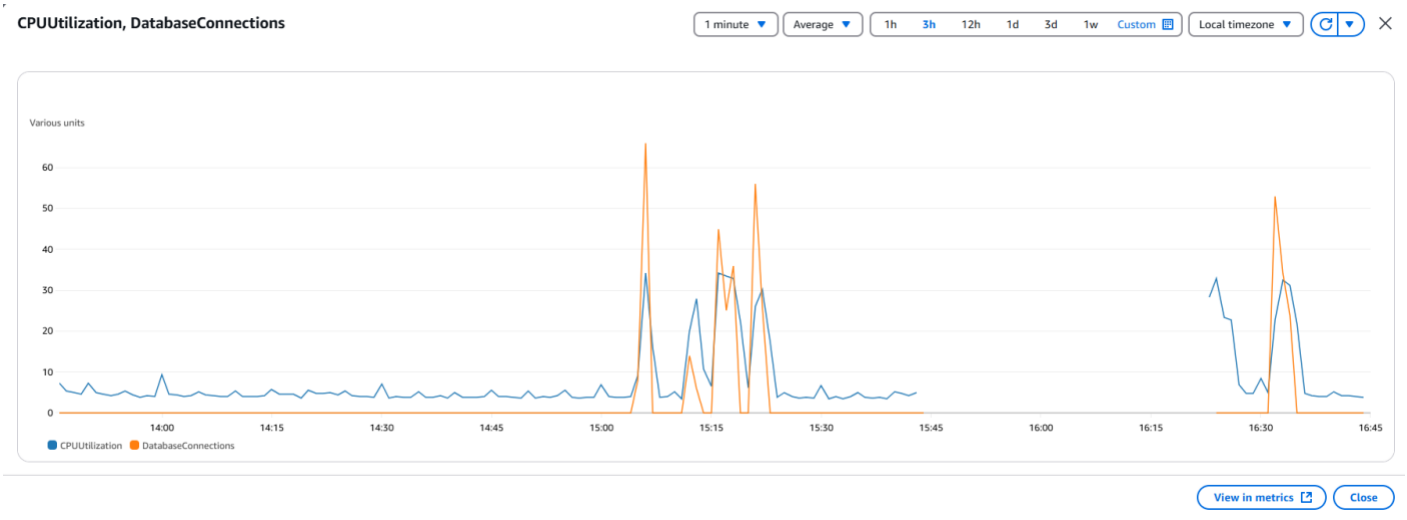
  These events were recorded in the Auto Scaling activity log and verified in CloudTrail logs.



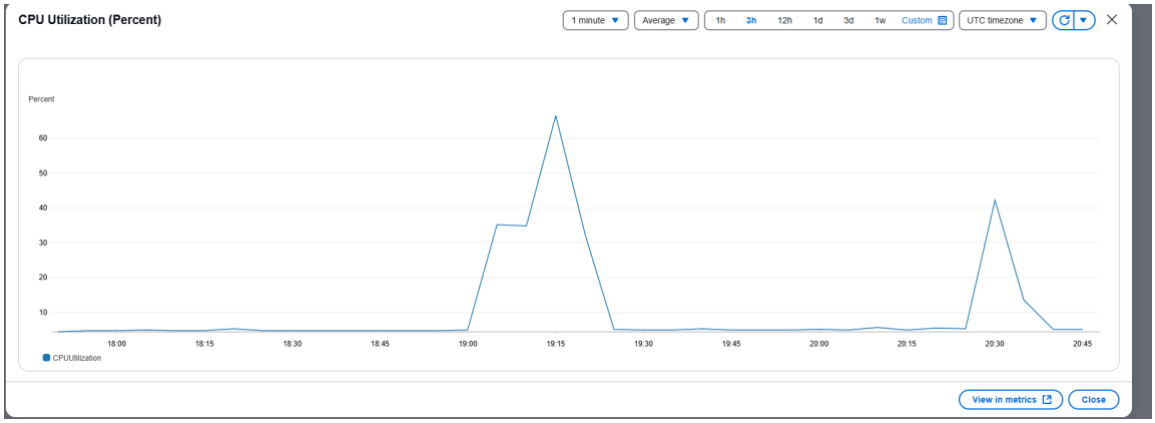


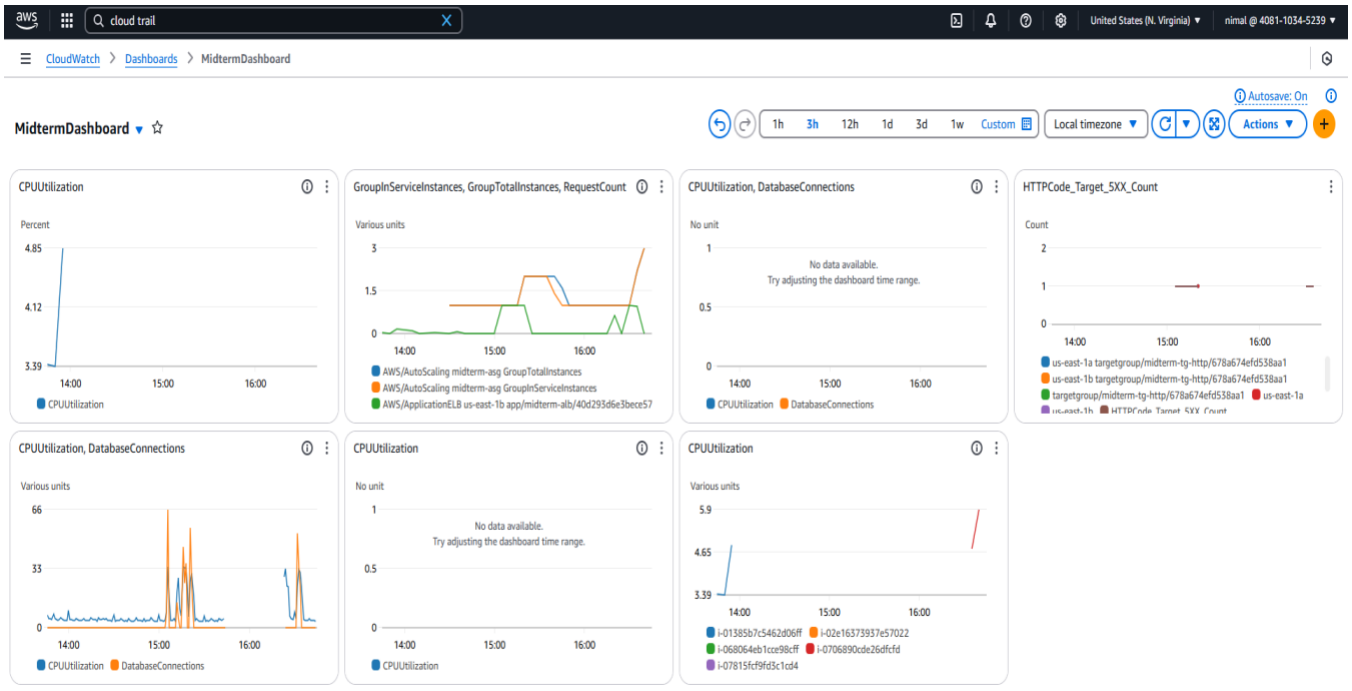

- CloudWatch Monitoring Dashboard
  A comprehensive dashboard was created with widgets tracking:
  - EC2 instance-level CPU utilization
  - Auto Scaling Group instance counts as well as total number of instances
  - ALB HTTP error rates and target health
  - NetworkIn and NetworkOut metrics
  - RDS CPU, latency, and connection count
  - S3 and CloudFront throughput

  This dashboard enabled real-time visualization while the test was running and helped to correlate spikes in traffic with the relevant scaling and resource metrics.
  We used the dashboard to:
  - Verify scaling thresholds that were triggered
  - Observe CPU utilization rise on each instance as well as the fall
  - Confirm that database activity increased significantly during POST-heavy load
  - Track 5XX errors, which had remained quite low throughout




GroupInServiceInstances, GroupTotalInstances, RequestCount
5 minutes
Average
1h
3h
12h
1d
3d
1w
Custom
Local timezone
Various units
AWS/AutoScaling midterm-asg GroupTotalInstances
AWS/AutoScaling midterm-asg GroupInServiceInstances
AWS/ApplicationELB us-east-1b app/midterm-alb/40d293d6e3bece57 targetgroup/midterm-tg-http/678a674efd538aa1 RequestCount
View in metrics
Close

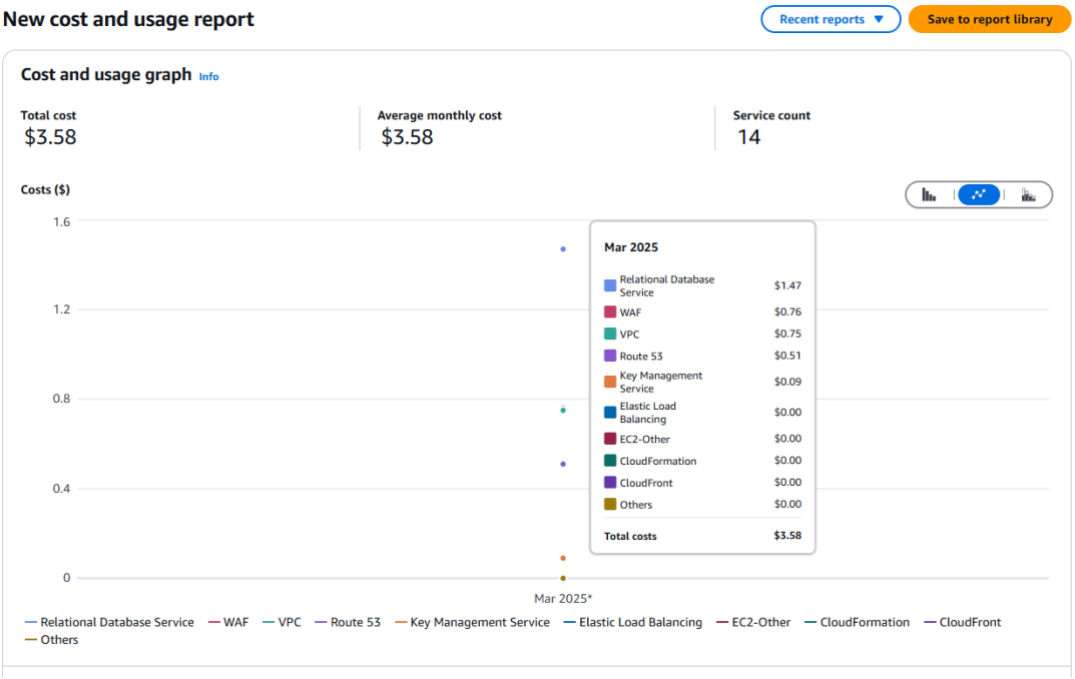

CPU Utilization (Percent)
1 minute
Average
UTC timezone
Percent
CPUUtilization
View in metrics
Close


cloud trail
United States (N. Virginia)
nimal @ 4081-1034-5239
CloudWatch
Dashboards
MidtermDashboard
Autosave: On
Actions
CPUUtilization
GroupInServiceInstances, GroupTotalInstances, RequestCount
CPUUtilization, DatabaseConnections
HTTPCode_Target_5XX_Count
No data available.
Try adjusting the dashboard time range.

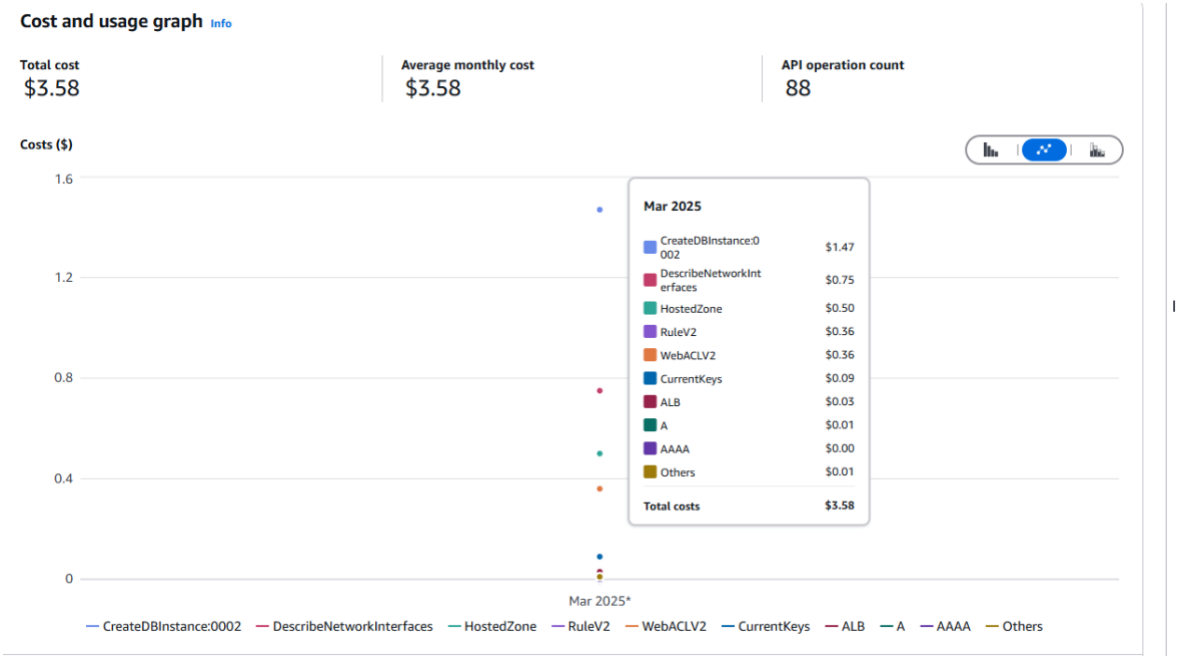
# 3. Cost Analysis & Optimization Recommendations

## a. Cost Analysis

According to the projected usage by our group for our infrastructure and our e-commerce platform, the EC2 instances and RDS accounted for the majority of billing cost of an estimate of 41%. This can align with the expectation that EC2 and RDS will demand as much according to average AWS resource usage for regular infrastructure. Monitoring tools and the WAF had minimal impact to the cost but is very essential for observability and security.

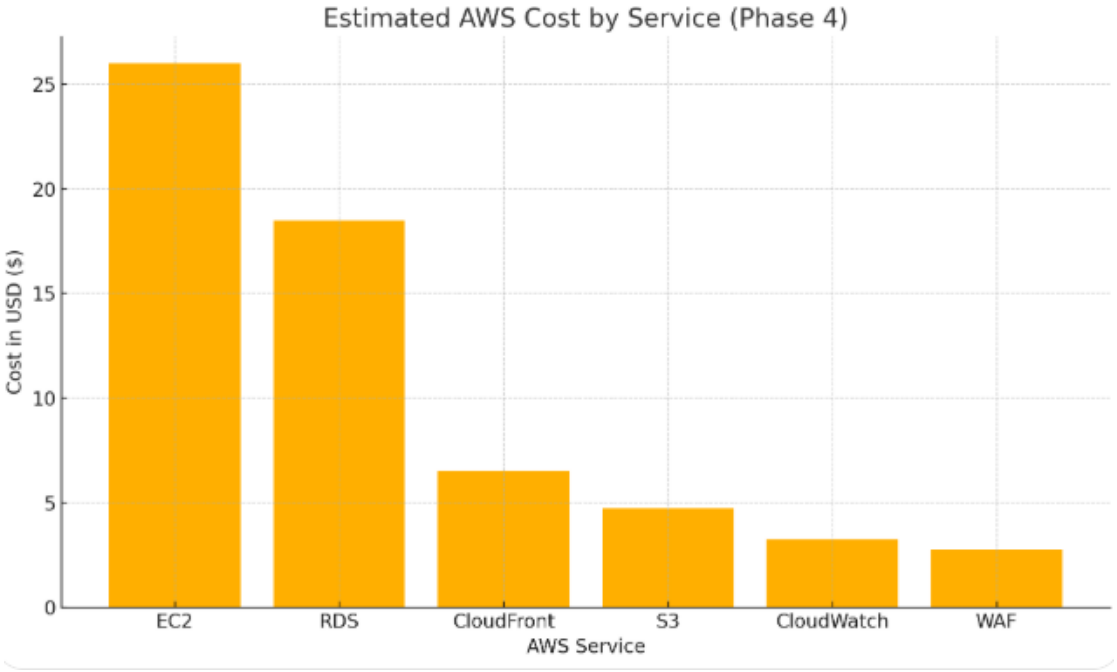



Usage by services:



Cost per API call:

Since it has not been long, a cost analyzer cannot provide accurate data but we can see more through an approximation given below:




### Cost Analysis via AWS Cost Explorer

We analyzed cost data for the current billing cycle. During the stress testing window, we noted:

- EC2 cost had increased due to multi-hour instance provisioning
- CloudWatch costs increased metrics collection and logs and the same was reflected
- RDS usage slightly increased due to significantly higher IOPS
- S3 and CloudFront costs remained low due to effective caching which did not trigger the use of the RDS and object tiering

No major cost anomalies were detected. However, this exercise revealed areas for savings through usage rightsizing and commitments.

## b. Optimization Recommendations

1. EC2 Reserved Instances: Converts the baseline EC2 capacity to Reserved Instances to cut cost by a margin of at least 30%.
2. Spot Instances for scale-out: Use Spot Instances for temporary scale-out to, this can reduce variable charges.
3. Database optimization: Enable slow query logging so as to identify high-latency queries and tune appropriately.
4. CloudWatch log retention: Reduce retention up to 7 days.
5. Enable GZIP and caching: Increase compression for dynamic content for faster delivery and less reads to the RDS.
6. Move asset delivery to CloudFront + S3: Offload load from the EC2 instance.
7. Predictive scaling: Train Auto Scaling on historical traffic patterns for instance provisioning.

# 4. Lessons Learned and Future Improvements

## a. Key Challenges

1. EC2 Auto Scaling Configuration
   - Ensuring the Auto Scaling Group properly scales up and down based on CPU utilization without unnecessary resource consumption.
   - Avoiding over-provisioning, which could lead to higher AWS costs.
2. Database Performance and Availability
   - Managing database connections efficiently to prevent bottlenecks.
   - Ensuring proper failover handling in Multi-AZ RDS deployment.
3. Security Configuration
   - Ensuring EC2 security groups and IAM roles follow the principle of least privilege.
4. Balancing GET/POST Requests for Realistic Load Simulation
   - GET requests dominate typical traffic but do not fully test backend performance.
   - POST requests are essential for evaluating database writes and overall backend health.
5. Target Tracking vs. Step Policies in Auto Scaling
   - Target tracking is easier to manage but requires fine-tuning to prevent over/under-scaling.
   - Misconfigured tracking policies may lead to delayed scaling responses.
6. RDS as a Performance Bottleneck
   - High database load due to inefficient queries or insufficient instance sizing.
   - Need for constant monitoring to prevent slow queries and resource exhaustion.
7. Scaling Behavior and Load Balancer Optimization
   - Scaling effectiveness depends on accurate warm-up periods for EC2 instances.
   - Traffic distribution from the ALB needs optimization to avoid overloading specific instances.
8. Observability and Monitoring Challenges
   - Lack of proper logging and metrics can obscure performance issues.
   - Monitoring tools must be actively used to detect issues before they impact users.
9. Content Compression in CloudFront
   - To ensure both CloudFront's "Compress Objects Automatically" setting and origin server's Content-Encoding headers are properly configured.

## b. Recommendations

1. Database Performance Best Practices
   - Enable RDS Performance Insights to analyze and optimize query execution.
   - Use Read Replicas for better performance if read-heavy workloads are expected.
2. Security Best Practices
   - Use IAM roles instead of hardcoded credentials in application code.
   - Implement AWS Shield for additional protection against DDoS attacks.
3. Simulate a Realistic Workload in Load Testing
   - Balance GET and POST requests in load tests to reflect actual user interactions.

- Use tools like Apache JMeter or AWS Load Testing to analyze backend performance.

4. Optimize Target Tracking Scaling Policies
   - Regularly adjust CPU utilization thresholds based on traffic patterns.
   - Consider hybrid scaling: Target Tracking for baseline stability and Step Scaling for rapid response to traffic spikes.
5. Improve RDS Performance and Availability
   - Enable RDS Performance Insights and query optimization for slow queries.
   - Consider Read Replicas for scaling read-heavy workloads.
   - Use RDS Proxy to manage database connections more efficiently.
6. Fine-Tune Warm-up Periods for EC2 Scaling
   - Set warm-up periods based on historical startup time metrics to prevent premature scaling decisions.
   - Test different ALB traffic routing strategies (e.g., Least Outstanding Requests) for better load distribution.
7. Enhance Observability with AWS Monitoring Tools
   - Set up CloudWatch Dashboards for CPU, memory, and database metrics.
   - Use CloudTrail to track API activity and security events.
   - Implement AWS X-Ray for end-to-end request tracing.
8. AWS Certificate Manager
   - Request Certificate status was pending validation for several hours, CNAME entry in route 53 was added manually to resolve the issue.
9. Set Appropriate TTL Values
   - Configure CloudFront's Minimum, Default, and Maximum TTL settings to control how long content stays cached, balancing performance with content freshness.

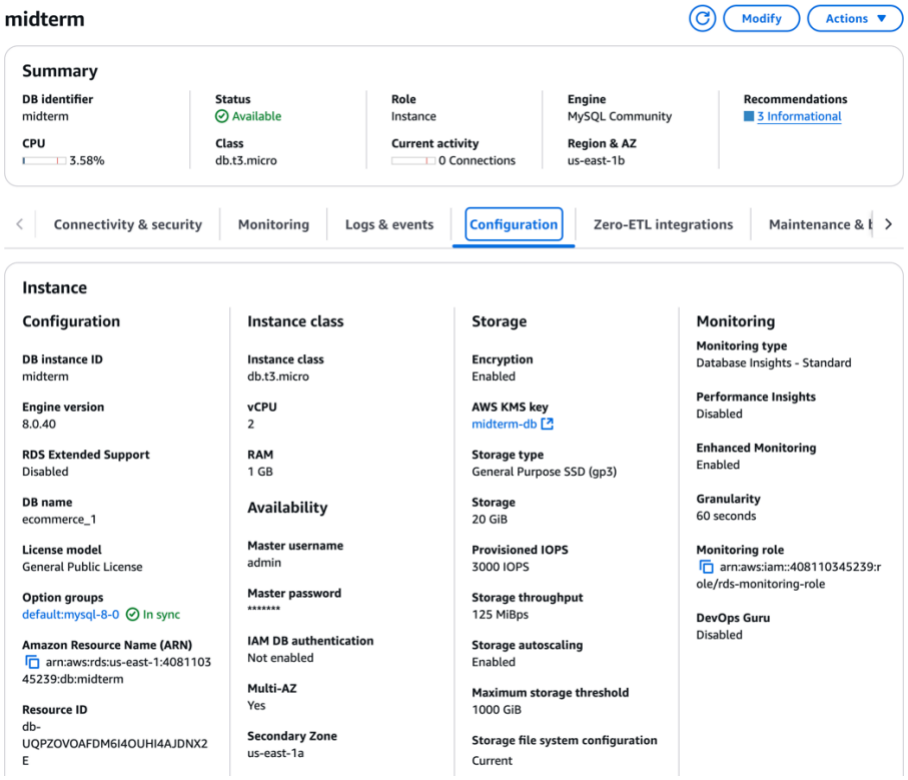
### c. Future Enhancements

1. Implement Containerization
   - Migrate EC2-based deployment to Amazon ECS or EKS to improve scalability and management.
2. Advanced Auto Scaling
   - Use Predictive Auto Scaling to anticipate traffic patterns and scale accordingly.
3. Advanced Load Testing Strategies
   - Implement a continuous load testing pipeline to assess system performance under various traffic conditions.
   - Automate testing using AWS Distributed Load Testing.
4. Database Performance Scaling
   - Integrate ElastiCache (Redis/Memcached) to reduce direct database queries.
   - Use Aurora Serverless for auto-scaling database capacity during demand spikes.
5. Intelligent Auto Scaling with Machine Learning
   - Implement Predictive Scaling to anticipate traffic patterns and optimize scaling decisions.
6. Infrastructure as Code (IaC) for Better Maintainability
   - Use AWS CloudFormation or Terraform to define and manage infrastructure consistently.

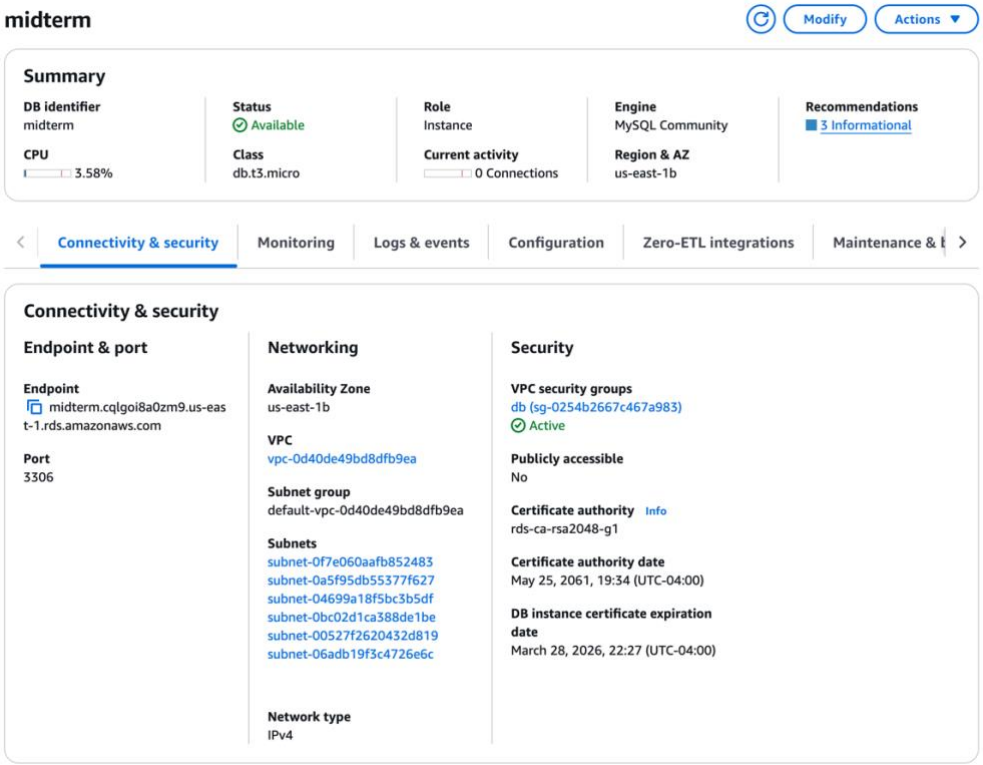
7. Security and Compliance Enhancements
    - Enable AWS GuardDuty for real-time threat detection.
    - Implement AWS Config to enforce compliance with security best practices.
8. Leverage Origin Shield:
    - Enable CloudFront's Origin Shield to add an extra caching layer that reduces origin server load and enhances cache hit ratios.
9. **Redirect Overhead:**
    - The redirection introduces additional latency due to the extra round-trip between the client and the ALB before reaching CloudFront.

# 5. Screenshots of infrastructure deployment

1. Database: Configuration




2. Database: Connectivity & Security



## 3. AMI Creation

**Image summary for ami-0930d4891cbcb9a1d**

EC2 Image Builder | Actions | Launch instance from AMI

| | | | |
|---|---|---|---|
| **AMI ID**<br>ami-0930d4891cbcb9a1d | **Image type**<br>machine | **Platform details**<br>Linux/UNIX | **Root device type**<br>EBS |
| **AMI name**<br>midterm-ami-v3 | **Owner account ID**<br>408110345239 | **Architecture**<br>x86_64 | **Usage operation**<br>RunInstances |
| **Root device name**<br>/dev/sda1 | **Status**<br>Available | **Source**<br>408110345239/midterm-ami-v3 | **Virtualization type**<br>hvm |
| **Boot mode**<br>uefi-preferred | **State reason**<br>– | **Creation date**<br>2025-03-29T03:02:11.000Z | **Kernel ID**<br>– |
| **Description**<br>updated db url | **Product codes**<br>– | **RAM disk ID**<br>– | **Deprecation time**<br>– |
| **Last launched time**<br>– | **Block devices**<br>/dev/sda1=snap-0db6bbd474e9d822a:8:true:gp3<br>/dev/sdb=ephemeral0<br>/dev/sdc=ephemeral1 | **Deregistration protection**<br>Disabled | **Allowed image**<br>- |
| **Source AMI ID**<br>ami-0c83bebe0f33b57c1 | **Source AMI Region**<br>us-east-1 | | |

## 4. Auto-Scaling Group Launch Template

**midterm-asglt (lt-0b9c99df0e0804dd7)**

Actions | Delete template

**Launch template details**

| | | | |
|---|---|---|---|
| **Launch template ID**<br>lt-0b9c99df0e0804dd7 | **Launch template name**<br>midterm-asglt | **Default version**<br>3 | **Owner**<br>arn:aws:iam::408110345239:user/harsha |

Details | Versions | Template tags

**Launch template version details**

Actions | Delete template version

| | | | |
|---|---|---|---|
| **Version**<br>3 (Default) | **Description**<br>new db url | **Date created**<br>2025-03-29T03:05:02.000Z | **Created by**<br>arn:aws:iam::408110345239:user/harsha |

Instance details | Storage | Resource tags | Network interfaces | Advanced details

| | | | |
|---|---|---|---|
| **AMI ID**<br>ami-0930d4891cbcb9a1d | **Instance type**<br>t2.micro | **Availability Zone**<br>- | **Key pair name**<br>midterm |
| **Security groups**<br>- | **Security group IDs**<br>sg-07f1f54f3fb32ff89 | | |

## 5. Auto-Scaling Group: Configuration

**midterm-asg**

**midterm-asg Capacity overview** — Edit

arn:aws:autoscaling:us-east-1:408110345239:autoScalingGroup:db1d025a-eba2-43d5-92e9-a100d1a3ee89:autoScalingGroupName/midterm-asg

| Desired capacity | Scaling limits (Min - Max) | Desired capacity type | Status |
|---|---|---|---|
| 2 | 1 - 3 | Units (number of instances) | Updating capacity |

**Date created**
Tue Mar 25 2025 14:26:27 GMT-0400 (Eastern Daylight Time)

Details | Integrations - *new* | Automatic scaling | Instance management | Instance refresh | Activity | Monitoring

**Launch template** — Edit

| Launch template | AMI ID | Instance type | Owner |
|---|---|---|---|
| lt-0b9c99df0e0804dd7 midterm-asglt | ami-0c83bebe0f33b57c1 | t2.micro | arn:aws:iam::408110345239:user/harsha |
| **Version** Default | **Security groups** - | **Security group IDs** sg-07f1f54f3fb32ff89 | **Create time** Tue Mar 25 2025 14:05:43 GMT-0400 (Eastern Daylight Time) |
| **Description** - | **Storage (volumes)** - | **Key pair name** midterm | **Request Spot Instances** No |

View details in the launch template console

**Network** — Edit

| Availability Zones | Subnet ID | Availability Zone distribution |
|---|---|---|
| us-east-1a, us-east-1b | subnet-04699a18f5bc3b5df, subnet-0f7e060aafb852483 | Balanced best effort |

## 6. Auto-Scaling Group: Automatic Scaling

**Dynamic scaling policies (2)** Info — Actions ▼ — Create dynamic scaling policy

< 1 >

**midterm-asg-dsp-simple**

**Policy type**
Simple scaling

**Enabled or disabled**
Enabled

**Execute policy when**
**midterm-asg-add-instance**
breaches the alarm threshold: GroupTotalInstances >= 3 for 1 consecutive periods of 300 seconds for the metric dimensions:
AutoScalingGroupName = midterm-asg

**Take the action**
Add 1 capacity units

**And then wait**
300 seconds before allowing another scaling activity

**Target Tracking Policy**

**Policy type**
Target tracking scaling

**Enabled or disabled**
Enabled

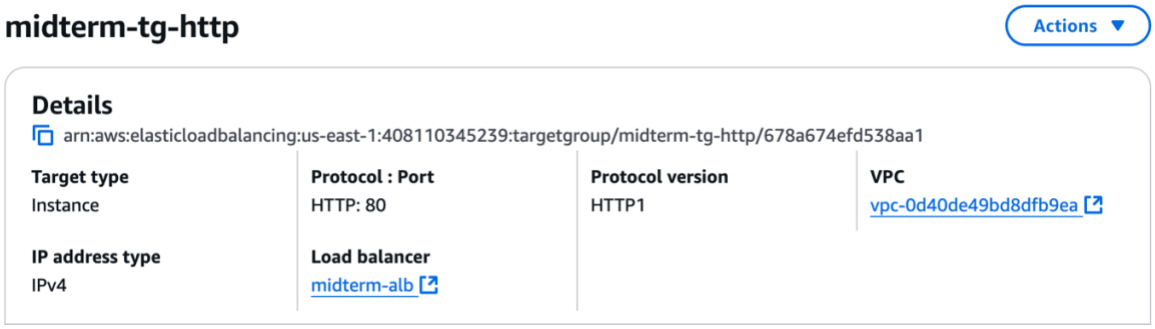
**Execute policy when**
As required to maintain Average CPU utilization at 50

**Take the action**
Add or remove capacity units as required

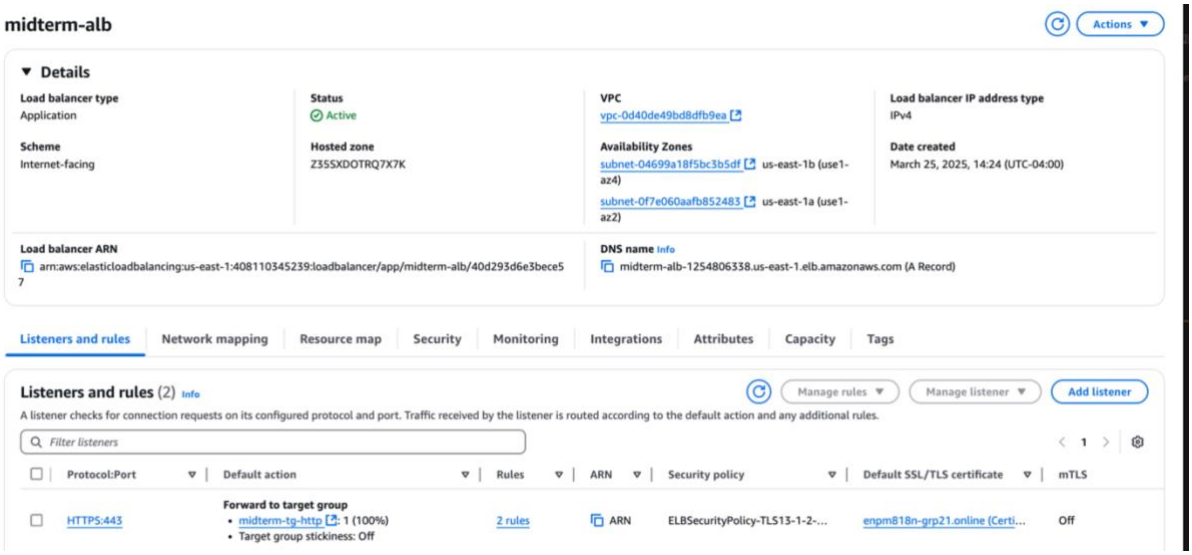
**Instances need**
300 seconds to warm up before including in metric

**Scale in**
Enabled

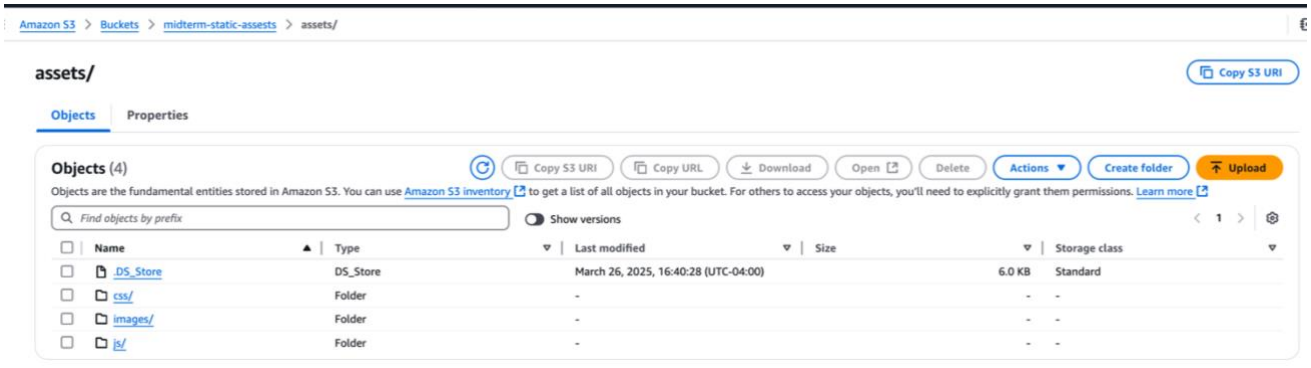
7. Application Load Balancer: Target Group



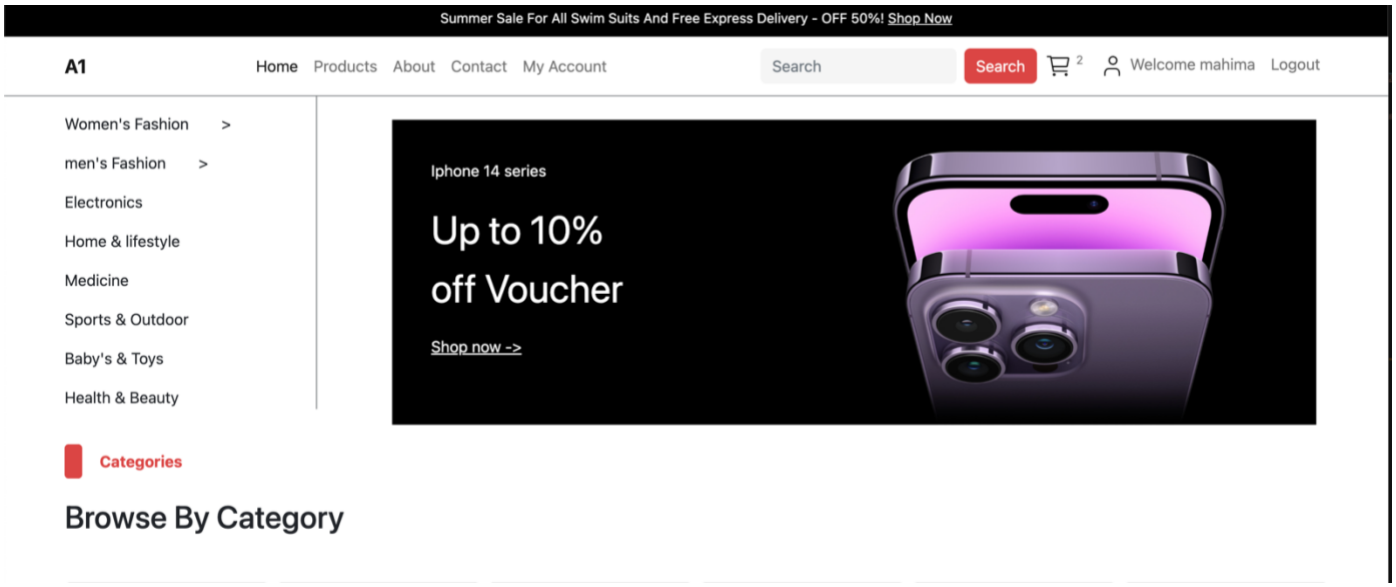

8. Application Load Balancer: Configuration



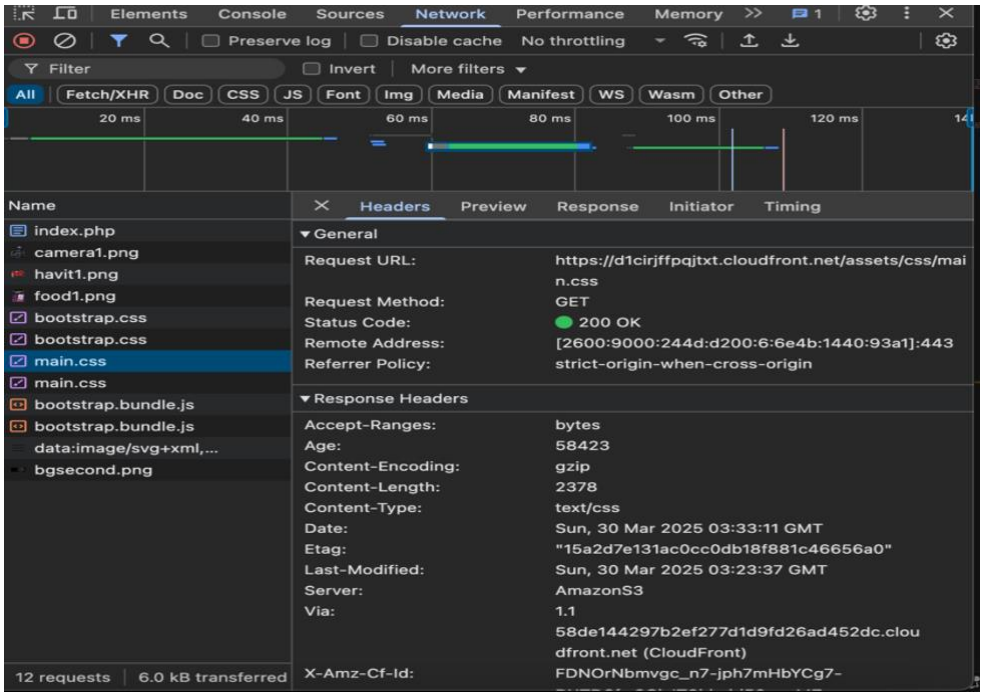

9. S3 Bucket for storing Static Assests



10. Able to Login as User




11. CloudFront Configuration



References:

https://aws.amazon.com/premiumsupport/support-cloud-cost-optimization/
https://docs.aws.amazon.com/awscloudtrail/latest/userguide/best-practices-security.html
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Introduction.html
https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_CreateDBInstance.html
https://docs.aws.amazon.com/acm/latest/userguide/certificate-validation.html
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-listeners.html
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/UsingWithRDS.SSL.html
https://docs.aws.amazon.com/waf/latest/developerguide/waf-rules.html
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/distribution-working-with.html